

MIT Art Design and Technology University
MIT School of Computing, Pune



Software Engineering and Project Management

Class - S.Y. (SEM-II),
Unit – I: Introduction to Software Engineering

AY 2022-2023 SEM-II



Course Objectives:

The main objectives of the course are to:

- **To learn and understand the principles of Software Engineering and understand Software Development Life Cycle**
- **To apply Project Management and Requirement analysis principles to S/W project development.**
- **To apply concepts of software engineering to develop UML diagrams in design.**
- **To understand the testing strategies and tools**
- **To understand the different risk factors & techniques of Software Project planning and management**

Course Outcomes:



On completion of the course, learner will be able to—

- 1. Compare and chose a process model for a software project development.**
- 2. Analyze and model software requirements of a software system.**
- 3. Design and Modeling of a software system with tools.**
- 4. Designing test cases of a software system.**
- 5. Prepare the SRS, Design document, Project plan and overall management of a given software system.**

Unit I - Syllabus



Unit I – Introduction to Software Engineering

09 hours

The Nature of Software, The Changing Nature of Software, Professional Software Development, Software Engineering Ethics, Software Development Myths, Software Process Models- the Waterfall Model, Incremental Process Models, Evolutionary Process Models, Concurrent Models, Personal and Team Process Models, coping with change, The Rational Unified Process.

What is Software ?

Software is a set of computer programs that provides functionality and performs desired tasks.

Technical Definition

Software is:

- 1. Instructions:** A computer programs which provides **desired functions** and **performance** while executed.
- 1. Data Structures:** Enables program to adequately (effectively) **manipulate** (handle) **information**.
- 1. Documents:** that describes **the operation** and **use of program**.

Software Examples

Examples

Operating Systems, Internet browser, Movie Player, Antivirus, Games etc...

Applications



What is Software Engineering ?

- Software engineering is the process of **designing, developing, testing, and maintaining** software.
- It is a **systematic and disciplined, quantifiable** approach to the **development, operation and maintenance** of software.

Software Processes



How the customer explained it



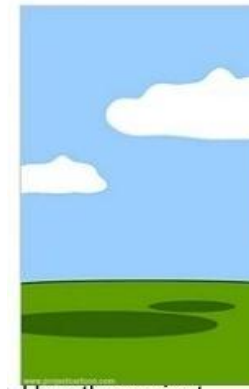
How the project leader understood it



How the analyst designed it



How the programmer wrote it



How the project was documented



What the beta testers received



What operations installed



What the customer really needed



How it performed under load



How the customer was billed

© <http://www.projectcartoon.com/>

The Evolving Role of Software

Dual Role of Software

1. A product

as an information transformer —

producing, managing, acquiring, modifying, displaying and/or transmitting information

2. The vehicle for delivering a product

as the **control of the computer** (Ex. operating systems),

the **communication of information** (Ex. Networking software) and the **creation of other programs** (Ex. software tools)

Software Applications

Software categorization based on their application areas

- 1) System Software
- 2) Business Software
- 3) Engineering and Scientific Software
- 4) Personal Computer Software
- 5) Web-based Software
- 6) Artificial Intelligence Software
- 7) Real-time System
- 8) Embedded System

System Software

Interacts with hardware and provides interface to other applications.

Examples

Operating System, Device Driver, Compiler, File Management Utilities etc...

Applications



04/02

Business Software

Software applications that are **used by business users** to perform various **business functions**.

Examples

Accounting System, Enterprise resource planning (ERP) etc...

Applications



05/02

Engineering and Scientific Software

Number crunching algorithms used to solve scientific problems

Examples

Matlab, Autocad etc...



Personal Computer Software

Prolific (productive) application domain

Examples

Word processing software, Spreadsheets, Multimedia Software etc...

Applications



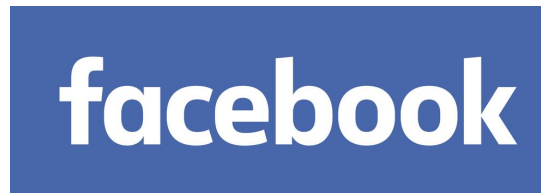
Web-based Software

The **web pages** retrieved by a web browser

Examples

Websites, Webpages

Applications



08/02

Artificial Intelligence Software

Computer systems able to perform tasks normally requiring human intelligence

Examples

Speech Recognition, Visual Perception, Decision-making and Translation between Languages etc..

Applications



Real-time System

Monitor, Analyse and Control the real time events.

Examples

Vehicle Event Controller, Radar, Air traffic control system etc...

Applications



Embedded System

Resides in read-only memory (ROM) of a device and is **used to control** its functions

Examples

Keypad Control for a Microwave Oven, Digital Functions in an Automobile such as Fuel Indicator, Dashboard Displays and Braking Systems

Applications



Software Characteristics

Software is a **logical** rather than a **physical** system (Physical Product).
Therefore, software product has **characteristics** that are considerably **different** than hardware (physical products)

1. Software is **developed or engineered**, it is **not manufactured** in the classical sense.
1. Software **doesn't "wear out."**
1. Although the industry is moving toward **component-based assembly**, most software continues to be **custom built**.

Hardware Vs. Software Failure

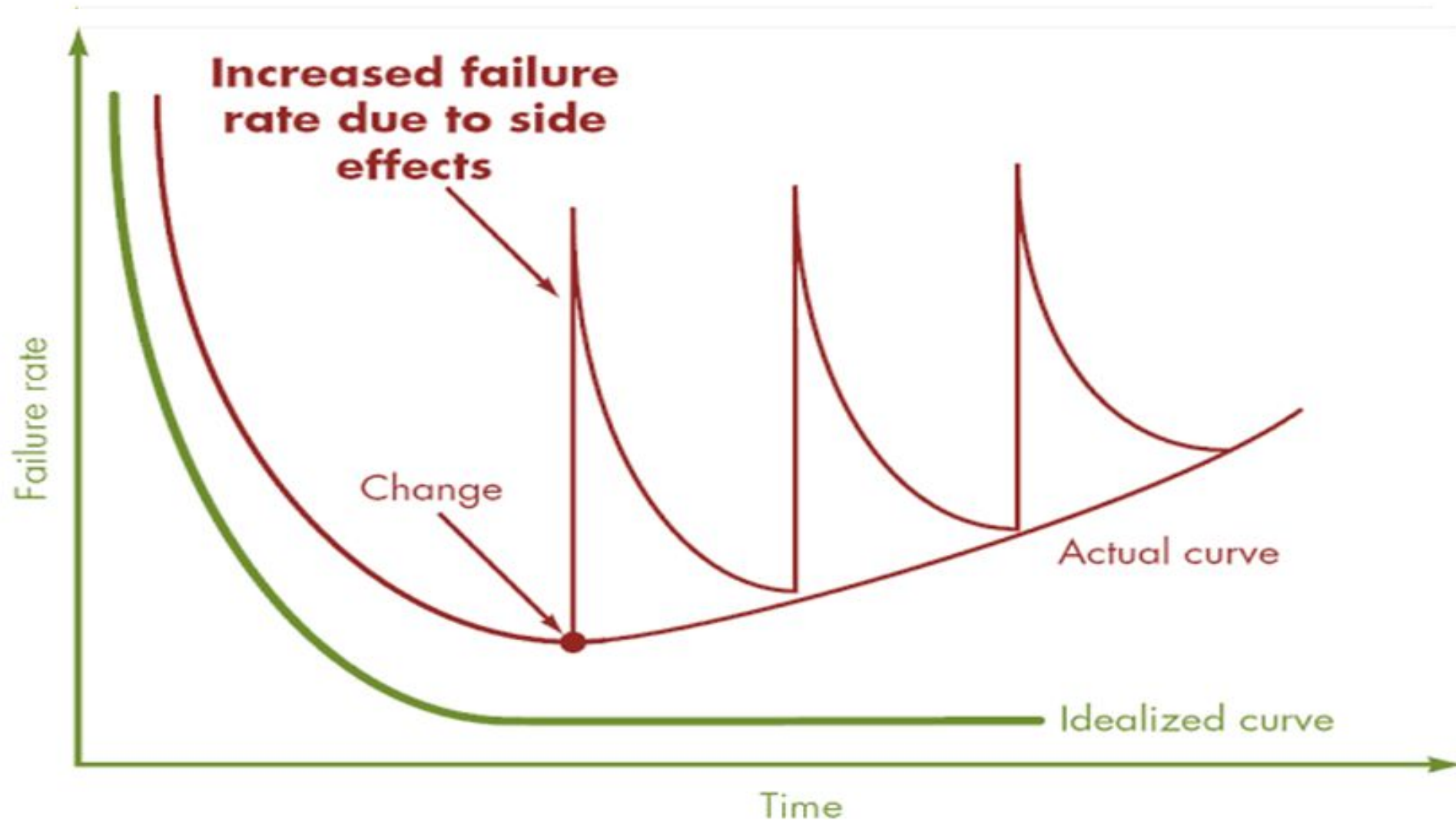
- Hardware faults are mostly physical faults.
- Software faults are design faults, which are tough to visualize, classify, detect, and correct.
- Hardware components generally fail due to wear and tear.
- Software component fails due to bugs.

Hardware Failure curve



03/03

Software Failure curve

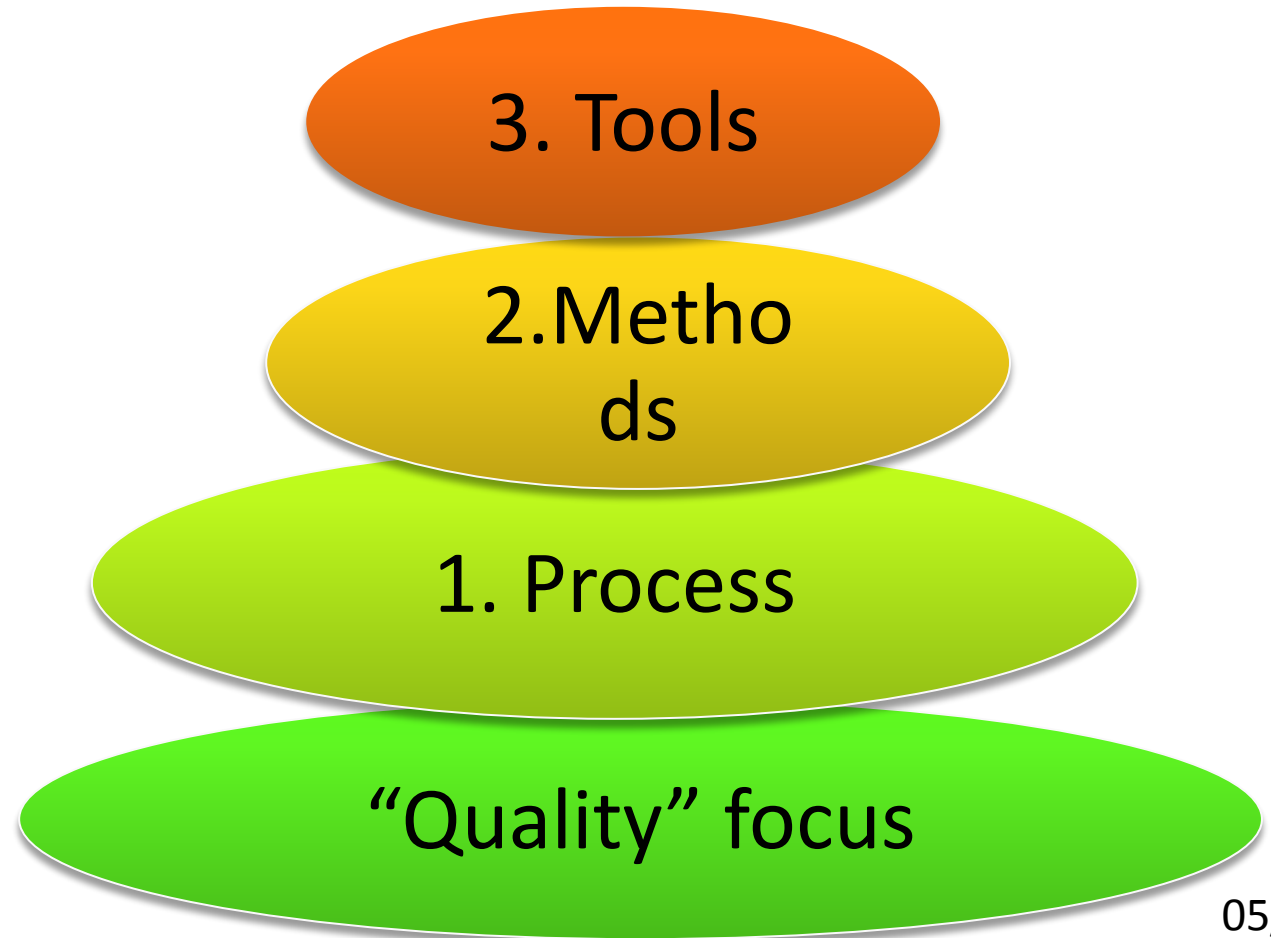


04/03

Software Engineering: A Layered Technology

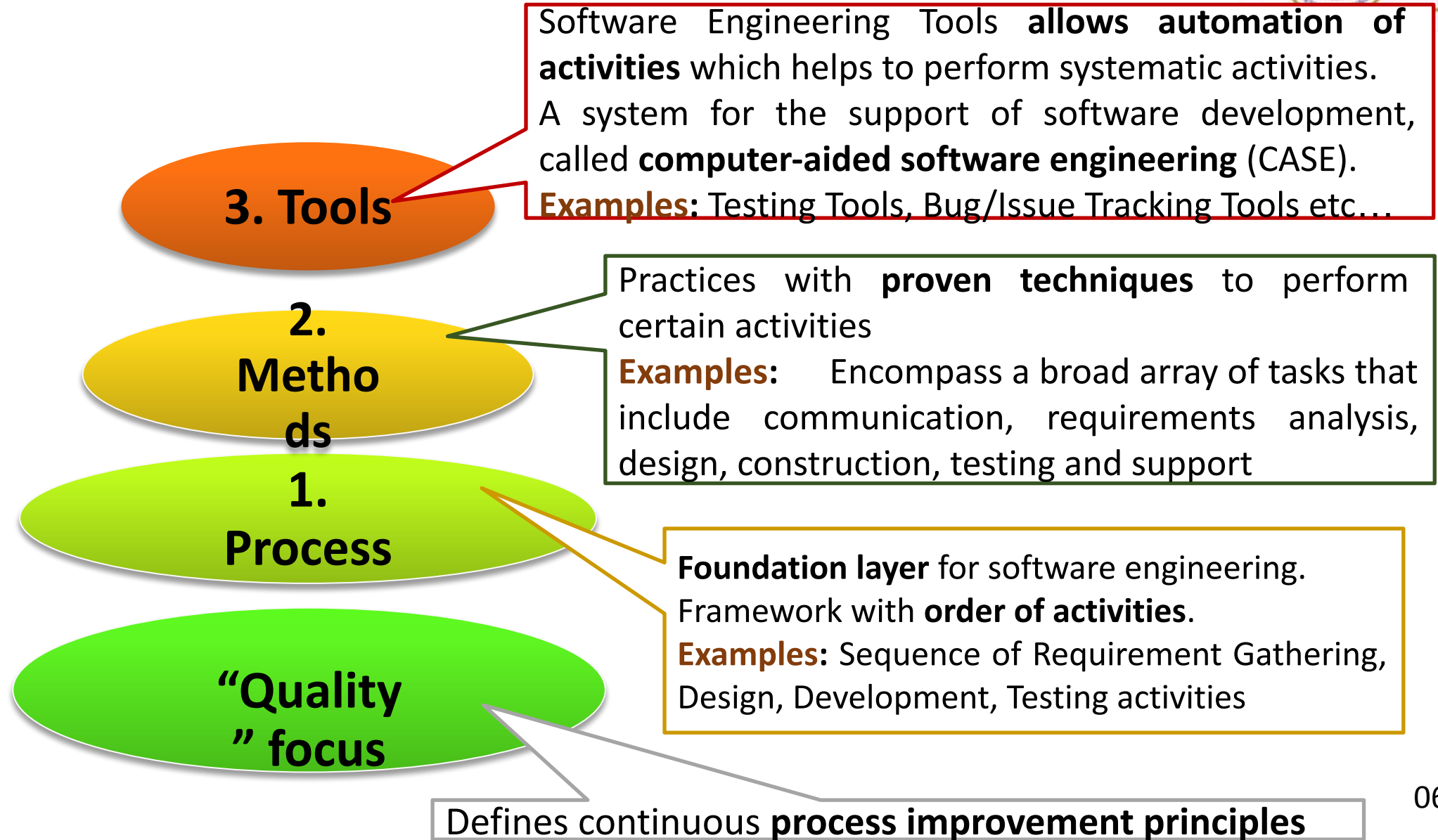
**Software engineering in practice
is built of 3 main layers**

- 1. Process**
- 2. Methods**
- 3. Tools**



05/03

Software Engineering: A Layered Technology



06/03

Software Myths

Beliefs about software and the process used to build it.

“Misleading Attitudes that cause serious problem” are myths.

I) Management Myths

I) Customer Myths

I) Practitioner's (Developer) Myths

Management Myths

Myth 1: We have **standards and procedures to build a system**, which is enough.

Reality:

- **practitioners aware**
- **reflect modern software engineering practices**
- **Is it complete?**
- **Is it streamlined to improve time to delivery while still maintaining a focus on quality?**

Myth 2: We have the **newest computers and development tools**.

Reality:

- **It takes much more than the latest model computers to do high-quality software development.**
- **Computer-aided software engineering (CASE) tools are more important than hardware.**

02/04

Management Myths

Myth 3: We **can add more programmers** and can catch up the schedule.

Reality:

- Software development **is not a mechanistic process** like manufacturing.
- In the words of Fred Brooks : "**adding people to a late software project makes it later.**"
- People who were working **must spend time educating the newcomers.**
- People can be added but **only in a planned and well-coordinated manner.**

Myth 4: I **outsourced the development activity**, now I can **relax** and can **wait** for the final working product.

Reality:

- If an organization does not understand how to manage and control software projects internally, it will invariably struggle when it outsources software projects.

Customer Myths

Myth 1: A general statement of objectives (requirements) is sufficient to start a development.

Reality:

- **Comprehensive** (detailed) **statements** of requirements is **not always possible**, an ambiguous (unclear) “statement of objectives” can lead to **disaster**.
- **Unambiguous** (clear) **requirements** can be gathered only through effective and continuous communication between customer and developer.

Myth 2: Requirement Changes can be easily accommodated because software is very flexible.

Reality:

- It is true that **software requirements change**, but the **impact of change varies with the time** at which it is introduced.
- When **requirements changes** are requested **early** the **cost impact** is relatively **small**.

Practitioner's (Developer) Myths

Myth 1: Once we write the program, our job is done.

Reality:

- Experts say "**the sooner you begin 'writing code', the longer it will take you to get done.**"
- Industry data indicates that 60 to 80 % effort expended on software will be after it is delivered to the customer for the first time.

Myth 2: I can't access quality until it is running.

Reality:

- One of the **most effective software quality assurance mechanisms** can be applied from the inception (beginning) of a project—**the technical review.**
- **Software reviews** are more effective “quality filter” **than testing** for finding software defects.

Practitioner's (Developer) Myths (Cont...)



Myth 3: Working program is the **only deliverable work product**.

Reality:

- A working program **is only one part** of a software configuration.
- A variety of work products (e.g., models, documents, plans) provide a foundation for successful engineering and, more important, guidance for software support.

Myth 4: Software engineering is about **unnecessary documentation**.

Reality:

- Software engineering is not about creating documents. It is about creating a **quality product**.
- Better quality leads to reduced rework. And reduced rework results in faster delivery times.

06/04

Software Process

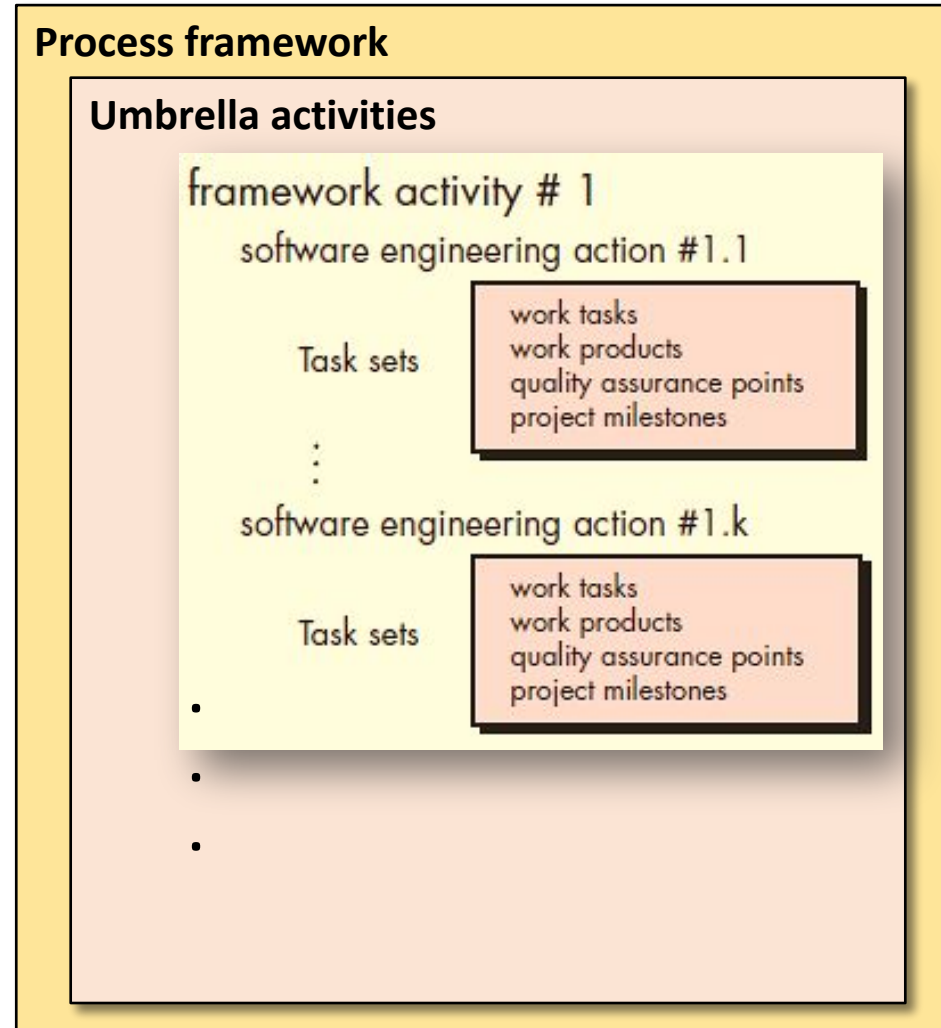
- 1. Communication**
- 2. Planning**
- 3. Modelling**
- 4. Construction**
- 5. Deployment**

Software Process Models

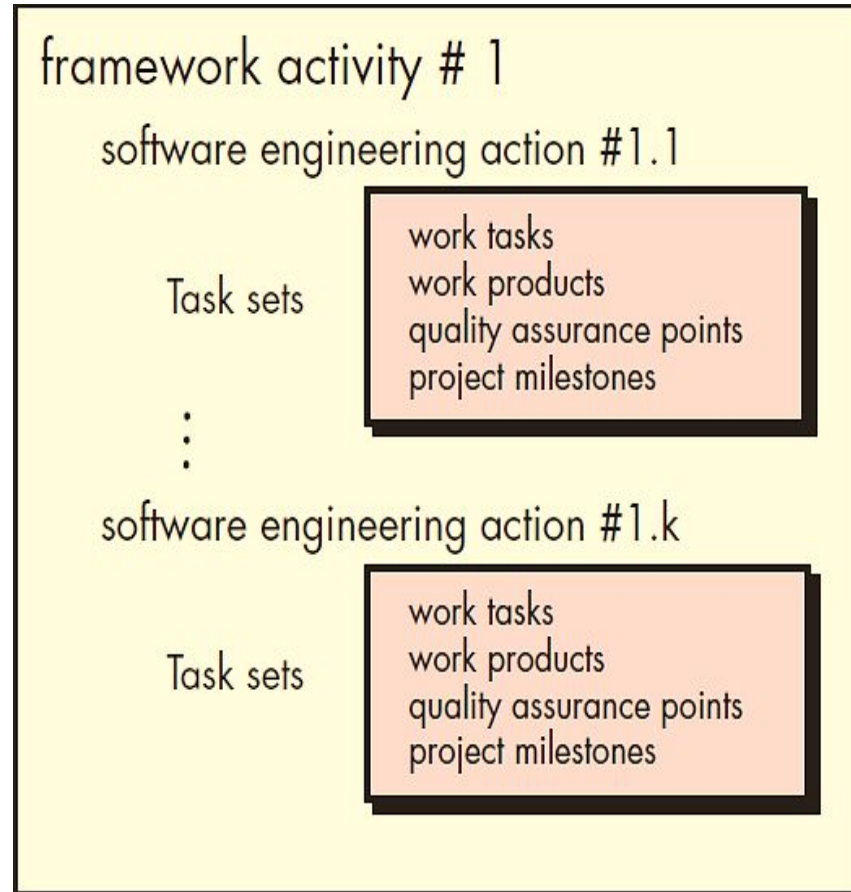
- The **process model** is the **abstract representation of process**.
- Also known as **Software development life cycle (SDLC)** or **Application development life cycle Models**
- Process models prescribe a distinct **set of activities, actions, tasks and milestones** (deliverables) required to engineer high quality software.
- Process models **are not perfect**, but **provide roadmap for software engineering work**.
- Software models provide **stability, control and organization** to a process that if not managed can easily get out of control.
- Software process models are adapted (adjusted) **to meet the needs of software engineers and managers** for a specific project.

Generic Process Model

Software process



Identifying a Task Set



A task set defines the **actual work** to be done to accomplish (achieve) the objectives of a software engineering action.

- A list of the **task** to be accomplished
- A list of the **work products** to be produced
- A list of the **quality assurance filters** to be applied
- A list of the **milestones** (special events) to determine project status

Generic Process Framework Activities

It encompasses (covers) five activities

1. Communication

- Communication with customers, stakeholders etc...
- Requirements gathering and related activities

2. Planning

- Defining workflow that is to follow
- Describe technical task, risks, resources requirements and a work schedule

3. Modelling

- Analysis of requirements
- Design of software

4. Construction

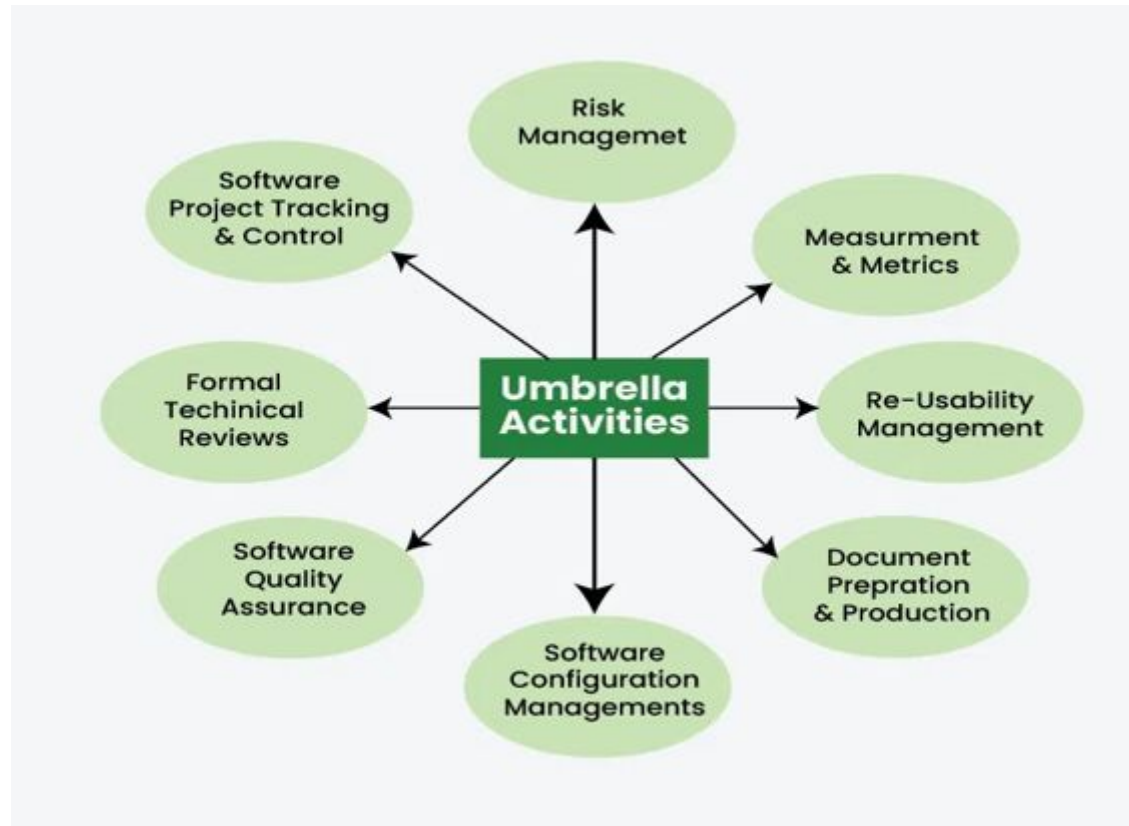
- Code generation
- Testing

5. Deployment

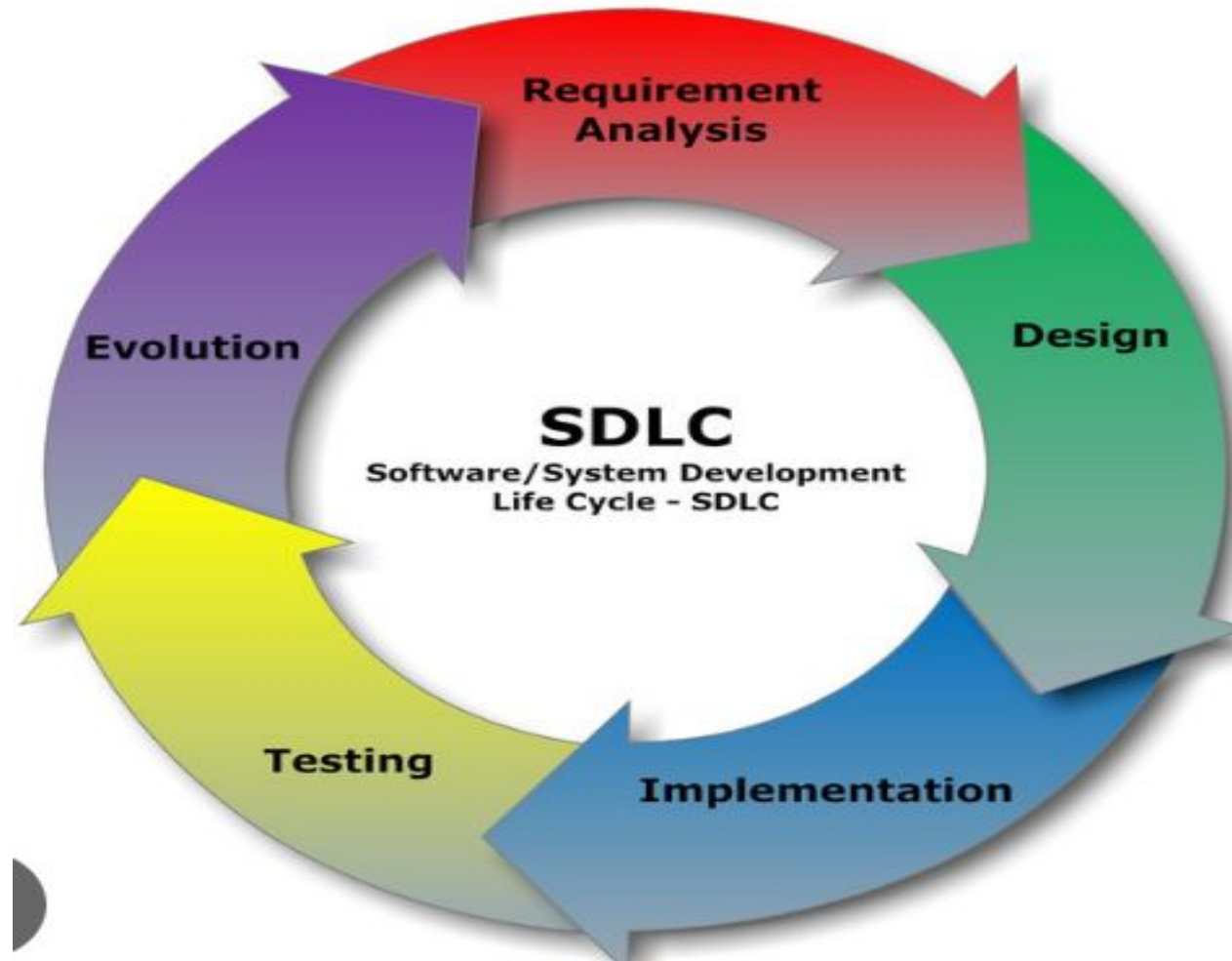
- Delivery to the customer
- Customer feedback
- Support

Umbrella Activities

Umbrella activities are **applied throughout a software project** and help a software team to **manage and control; progress, quality, change and risk**.

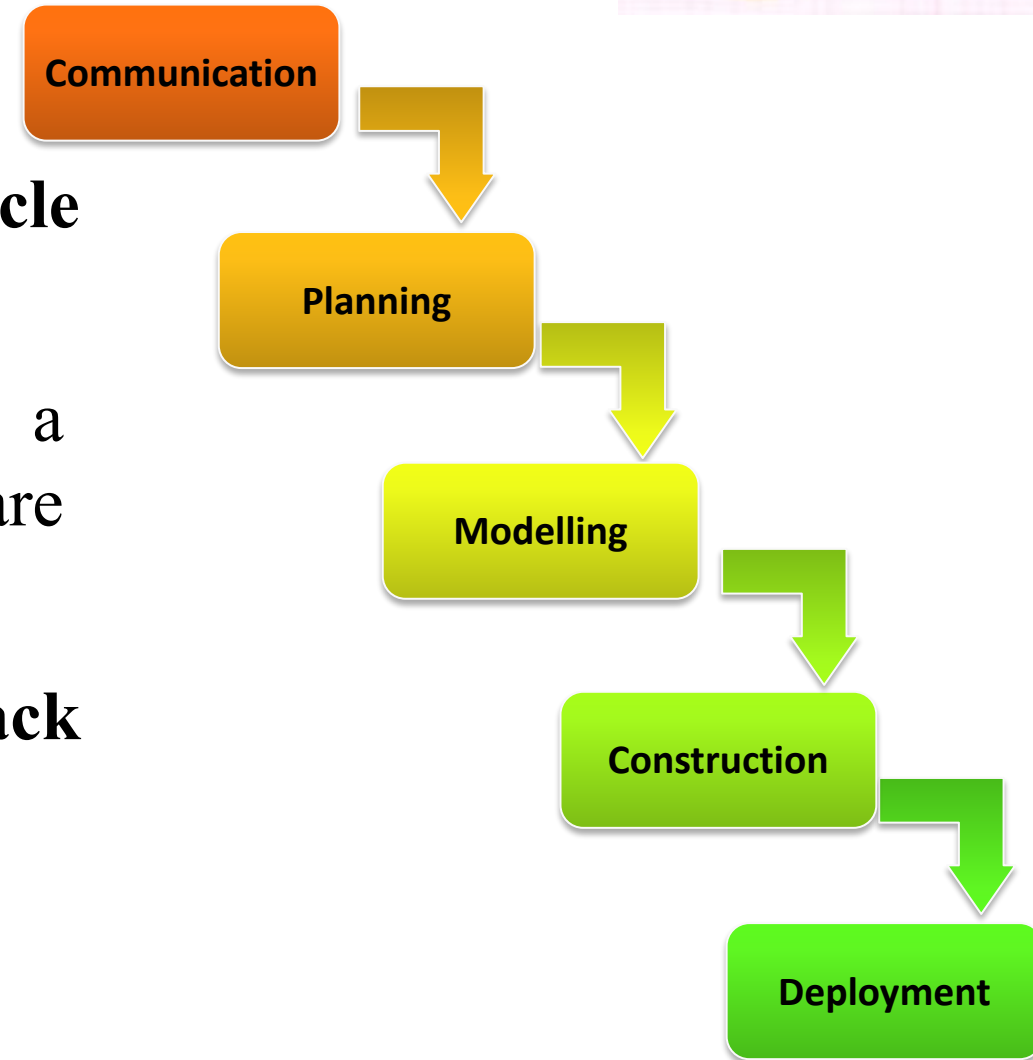


SDLC Phases



The Linear Sequential Model (Waterfall model)

- This Model also called as the **Classic life cycle** or the **Waterfall model**.
- **The Linear sequential model** suggests a systematic **sequential** approach to software development.
- Once a phase is complete, you **cannot go back** and repeat the process of previous phase.



The Linear Sequential Model (Cont...)

When to use ?

- **Requirements** are very well known, clear and fixed
- **Product definition** is stable
- **Technology** is understood
- There are **no ambiguous** (unclear) requirements
- Ample (sufficient) **resources** with required expertise are available freely
- The project is **short**

The Linear Sequential Model (Cont...)

Advantages

- **Simple** to implement and manage

Drawbacks

- Unable to **accommodate changes** at later stages, that is required in most of the cases.
- **Working version** is not available during development. Which can lead the development with major mistakes.
- **Deadlock** can occur due to delay in any step.
- Not suitable for **large projects**.

Incremental Model

- Waterfall model in **iterative manner**.
- **Initially core working product** is delivered.
- **For example**

word-processing software developed using the incremental model

- might deliver basic file management, editing and document production functions in the **first increment**;
- more sophisticated editing in the **second increment**;
- spelling and grammar checking in the **third increment**; and
- Advanced page layout capability in the **fourth increment**.

Incremental Model (Cont...)

When to Use ?

- The **requirements** of the complete system are **clearly defined** and understood.
- **Resources** with needed skill set are **not available** at the beginning.

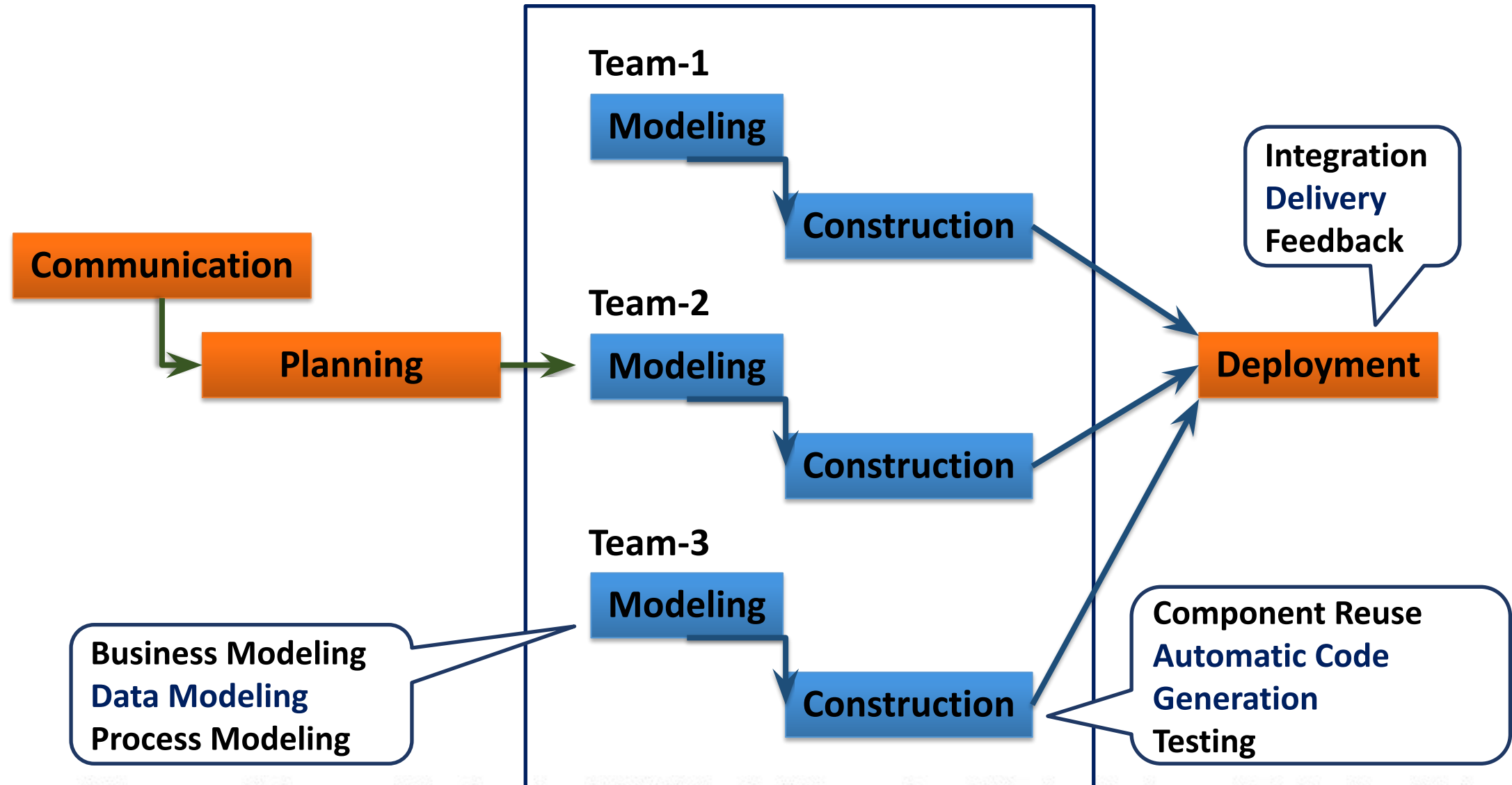
Advantages Incremental Model (Cont...)

- **Generates working software quickly** and early during the software life cycle.
- It is **easier to test** and **debug** during a smaller iteration.
- Customer can **respond to each built**.
- Lowers **initial delivery cost**.
- **Easier to manage risk** because risky pieces are identified and handled during iteration.

Disadvantages Incremental Model

- Needs **good planning** and **design**.
- Needs a **clear and complete definition** of the whole system before it can be broken down and built incrementally.
- Total **cost is higher** than Waterfall.

RAD (Rapid Application Development) Model



RAD Model (Cont...)

- It is a **type of incremental model** in which; components or functions are developed in **parallel**.
- It is achieved by component based construction
- This can **quickly** give the customer something to see and use and to provide feedback.

RAD Model (Cont...)

1. **Communication** to understand **business problem**.
1. **Planning** multiple software teams work in **parallel** on different systems/modules.
1. **Modeling**
 - 1) **Business Modeling:** Information flow among the business.
Ex. What kind of information? Who is going to generate information? From where information comes and goes?
 - 2) **Data Modeling:** Information refine into set of data objects that are needed to support business.
 - 3) **Process Modeling:** Data object transforms to information flow necessary to implement business.
2. **Construction** It highlighting the use of pre-existing software component.
1. **Deployment** Deliver to customer basis for subsequent iteration.

Advantages RAD Model (Cont...)

- Reduced **development time**.
- Increases **reusability** of components.
- Quick initial **reviews** occur.
- Encourages customer **feedback**.
- **Integration** from very beginning solves a lot of integration issues.

Disadvantages RAD Model (Cont...)



- For large but scalable projects, RAD **requires sufficient human resources**.
- Projects **fail if developers and customers are not committed** in a much shortened time-frame.
- Problematic if system can not be modularized.
- Not appropriate when **technical risks are high** (heavy use of new technology).

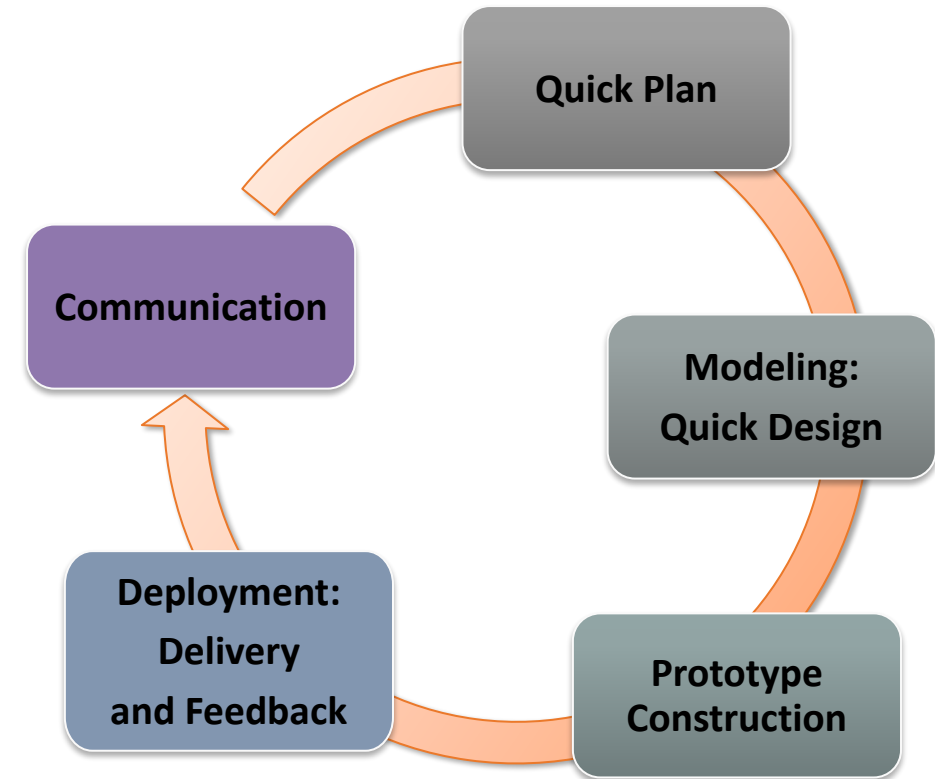
06/07

Evolutionary Process Models

- **Produce an increasingly more complete version** of the software with each iteration.
- Evolutionary Models are iterative.
- **Evolutionary models**
 - 1. **Prototyping Model**
 - 1. **Spiral Model**
 - 1. **Concurrent Development Model**

The Prototyping Model

- Serves as a **mechanism** for identifying software **requirements**.
- Prototype can be serve as “the **first system**”.
- Quick planning and modelling.
- Feedback from customer/end user will **refine** requirement.
- Both customers and developers like this paradigm.
 - Customer/End user **gets a feel** for the actual system.
 - Developer **gets to build** something immediately.



The Prototyping Model (Cont...)

When to Use ?

- Customers are not sure about the detail input, output and requirements.
- Developers are not sure about the technical feasibilities.

Problem Areas:

- Customer demand that “**a few fixes**” be applied to make the prototype a working product, due to that **software quality** suffers as a result.
- Developer often makes implementation in order to get a prototype working quickly; without considering other factors in mind like OS, Programming language, etc.

The Prototyping Model (Cont...)

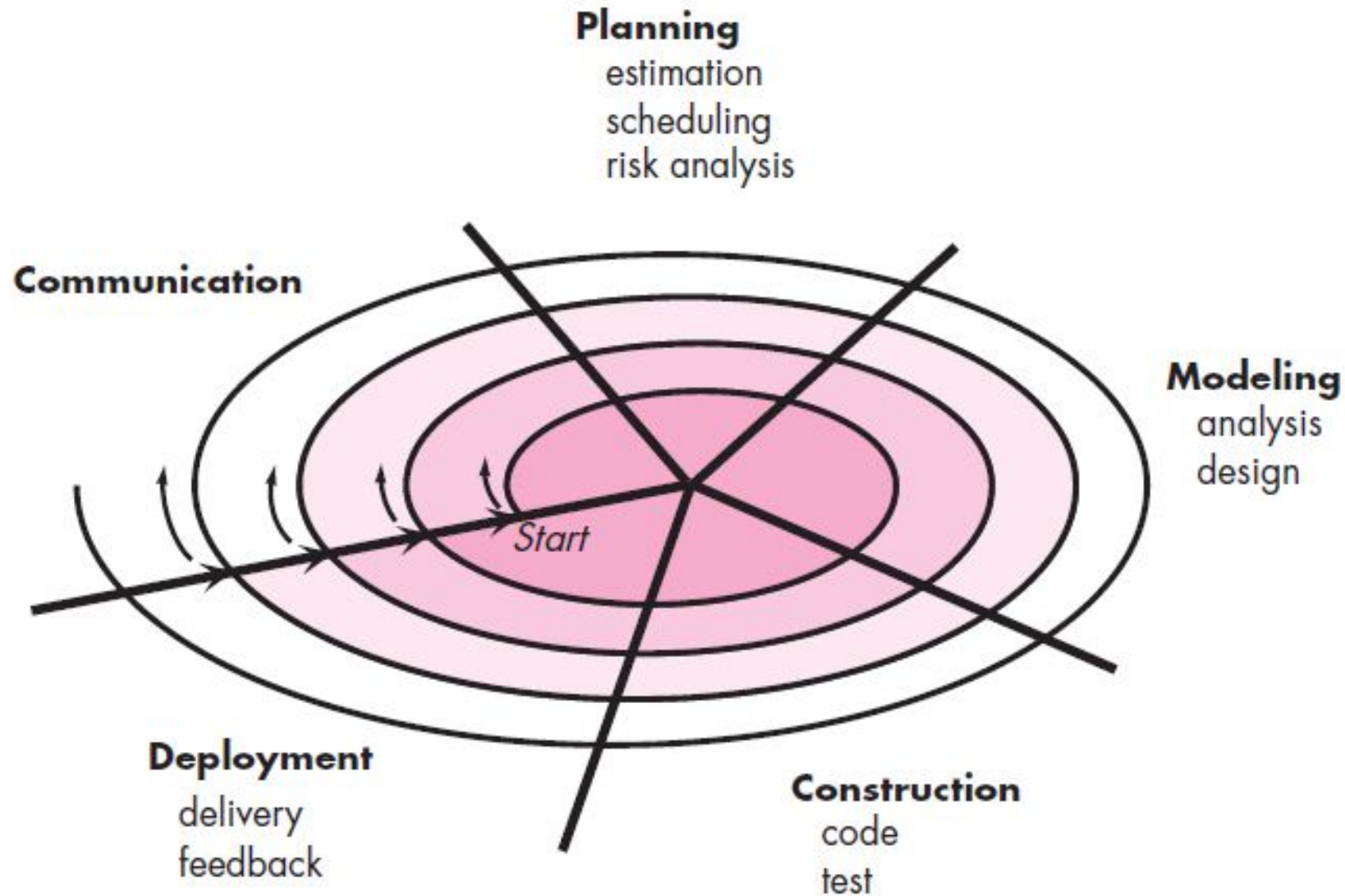
Advantages

- Users are **actively involved** in the development
- Since in this methodology a working model of the system is provided, the users **get a better understanding of the system** being developed.
- **Errors can be detected** much earlier.

Disadvantages

- Leads to implementing and then **repairing way of building** systems.
- Incomplete or inadequate (insufficient) problem analysis.

Spiral Model



05/08

Spiral Model (Cont...)

- Couples **iterative nature** of prototyping with the **controlled** and **systematic** aspects.
- Using spiral, software developed in a series of evolutionary release.
 - **Early iteration**, release might be on paper or prototype.
 - **Later iteration**, more complete version of software.
- Divided into framework activities **(C,P,M,C,D)**. Each activity represent **one segment**.

Spiral Model (Cont...)

When to use Spiral Model?

- When **costs** and **risk evaluation** is important.
- For **medium** to **high-risk** projects.
- Users are **unsure** of their **needs**.
- Requirements are **complex**.
- **New product** line.
- Significant (considerable) **changes** are **expected**

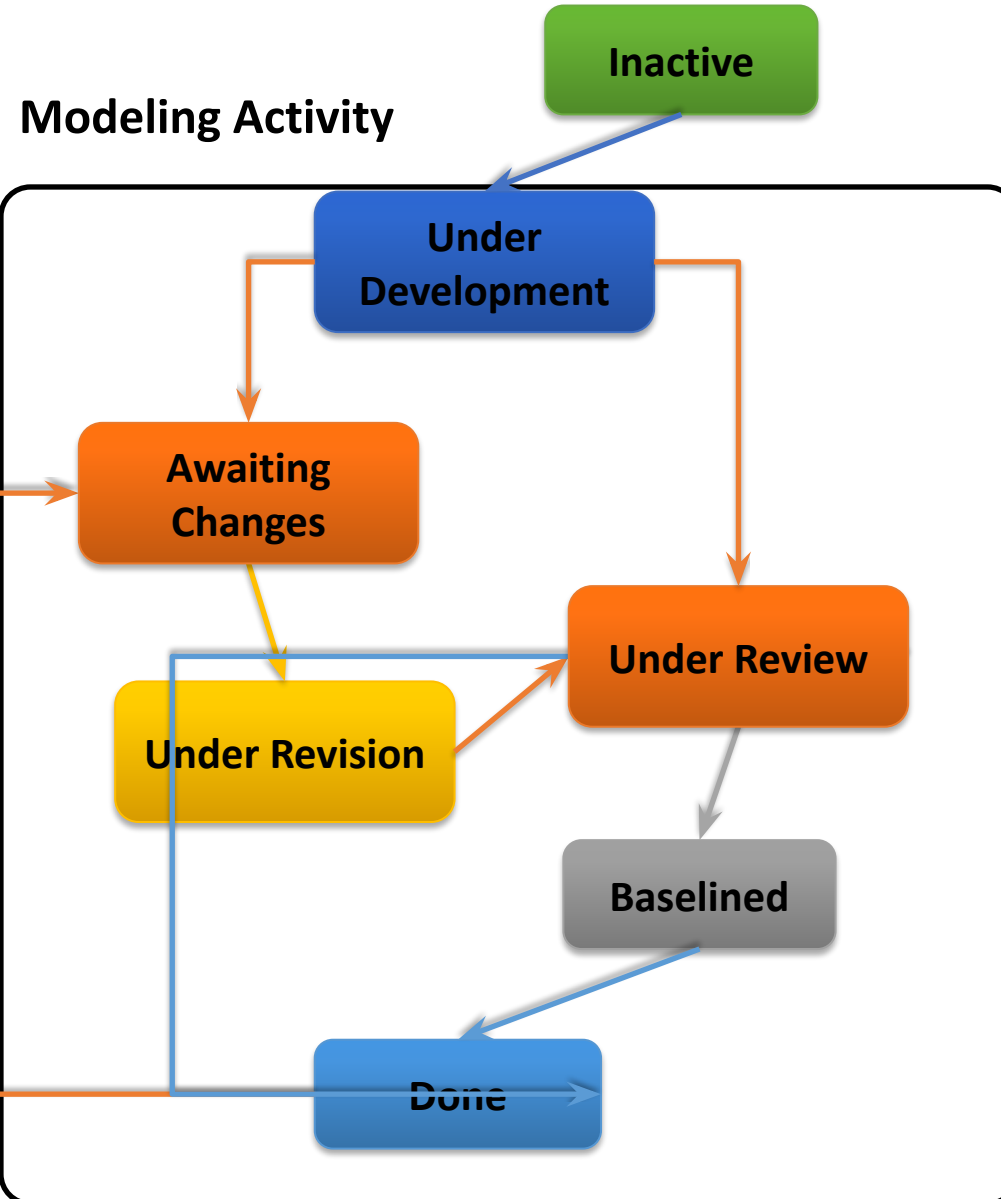
Advantages Spiral Model (Cont...)

- High amount of risk analysis hence, **avoidance of Risk** is enhanced.
- Good for **large projects**.
- Strong **approval** and **documentation control**.
- Additional **Functionality can be added** at a later date.
- **Software is produced early** in the Software Life Cycle.

Disadvantages Spiral Model (Cont...)

- Can be a **costly model** to use
- Risk analysis requires **highly specific expertise**.
- Project's success is highly dependent on the **risk analysis phase**.
- Doesn't work well for **smaller projects**.

Concurrent Development Model



- Allow a software team to represent **iterative and concurrent elements** of any of the process models.
- The Figure shows **modeling may be in any one of the states** at any given time.
- Each activity, action or task on the network exists **simultaneously** (all together) with other activities, actions or tasks.

Concurrent Development Model (Cont...)



- **Communication activity** has completed its first iteration and is in the **awaiting changes** state.
- The **modeling activity** was in inactive state, now makes a transition into the **under development** state.

Self Study

Agile Process Model

- Agile SDLC model is a **combination of iterative and incremental process models** with focus on process adaptability and customer satisfaction by rapid delivery of working software product.
- Agile Methods **break the product into small incremental builds**.
- Every iteration involves **functional teams working simultaneously** on various areas like planning, requirements analysis, design, coding, unit testing, and acceptance testing.

Agile Process Model



Self Study

Principles of Agile Software Development Methodology



- Achieving customer satisfaction by **timely** and **regular delivery** fitting the **customer requirements**
- **Delivering** working software with minimal or **no errors**
- Earning a competitive edge by **adapting** to the changes in the market
- Sticking to the **shortest time delivery** with **quality**
- Including **motivated** and **passionate** individuals in building and developing the project
- Accepting change in requirements and deliveries despite nearness to delivery date
- Choosing the face-to-face communication method for efficient and effective information disbursement
- Embracing excellence while estimating and improving the progress

Advantages Agile Process Model (Cont...)



- Customer satisfaction by **rapid, continuous delivery** of useful software.
- Customers, developers and testers **constantly interact with each other**.
- **Daily cooperation** between business people and developers.
- Continuous **attention to technical excellence** (quality) and **good design**.
- Regular **adaptation to changing circumstances**.

Self Study

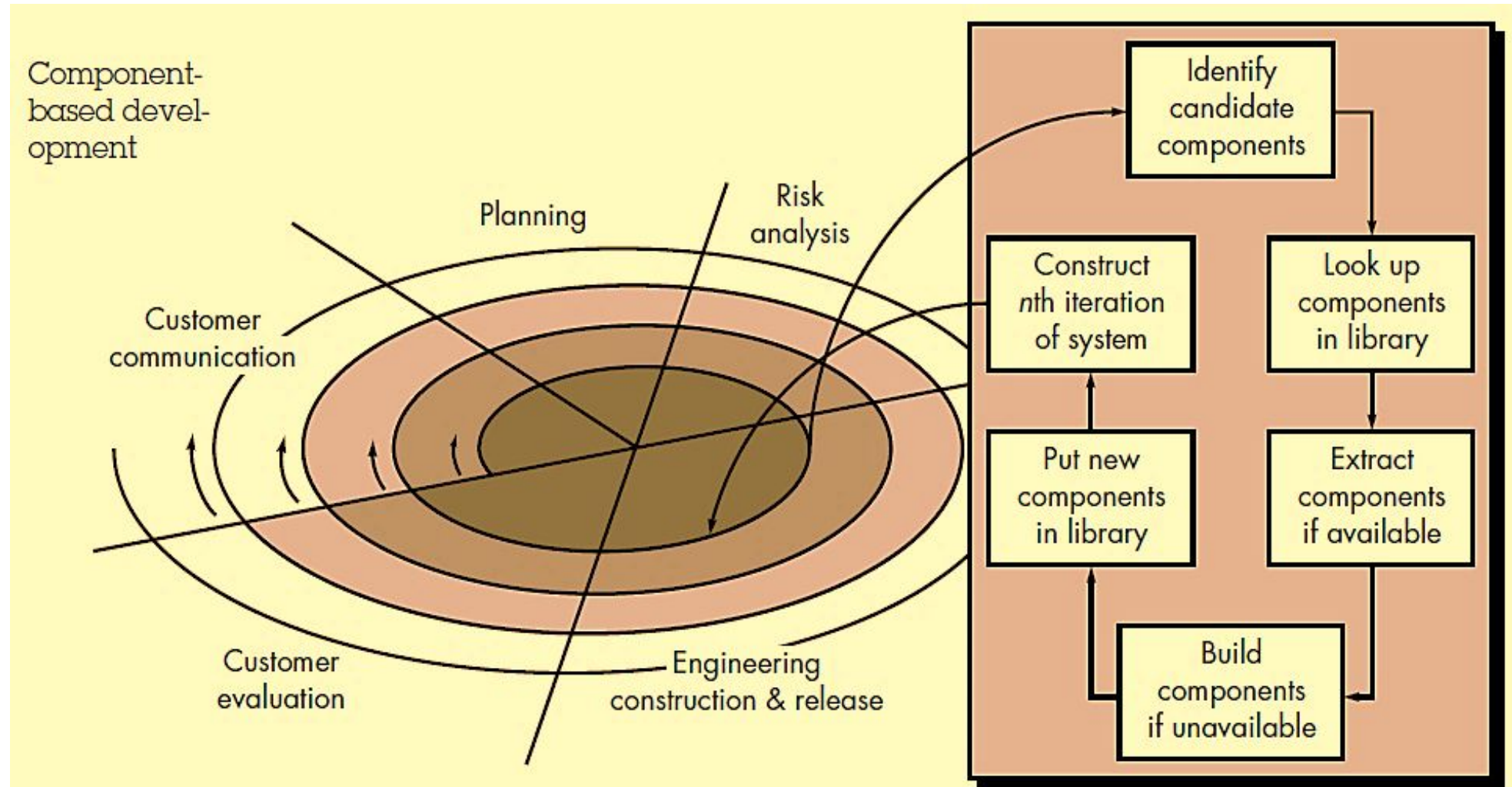
Disadvantages Agile Process Model (Cont...)



- In case of some software, it is **difficult to estimate the effort required at the beginning** of the software development life cycle.
- The project can easily get “**off track**” if the customer representative is not clear about final outcome, that they want.
- Only **senior programmers** are capable of taking the kind of decisions required during the development process.

Self Study

Component-Based Development



Self Study

Component-Based Development (Cont...)



- **Consists of the following process steps**
 - Available component-based products are **researched** and **evaluated** for the application domain.
 - Component **integration issues** are considered.
 - A **software architecture** is designed to accommodate the components.
 - Comprehensive (complete) **testing** is conducted to ensure proper functionality.
- Relies on a **robust** (healthy) **component library**.
- **Capitalizes** (take advantage) on **software reuse**, which leads to savings in project cost and time.

Self Study

Product and Process



- If the **process is weak**, the end product will suffer. But an overreliance (more confidence) on process is also dangerous.
- **People derive** (gain) as much (or more) **satisfaction** from the creative process as they do from the end product.
 - Like an artist **enjoys the brush strokes** as much as **the framed result**.
 - A writer **enjoys the search for the proper metaphor** (comparison) as much as **the finished book**.
- As creative software professional, you should also derive as much **satisfaction from the process as the end product**.
- The **duality** (contrast) of **product and process** is one important element in **keeping creative people engaged** as software engineering continues to evolve.

01/09

Extreme Programming (XP)

- Extreme Programming (XP) is an Agile software development methodology that focuses on delivering **high-quality software** through **frequent** and **continuous feedback**, **collaboration**, and **adaptation**.
- An XP project starts with user **stories** which are short descriptions of what **scenarios** the customers and users would like the system to support.



Principles of Extreme Programming (XP)

- XP is based on the frequent iteration through which the developers implement User Stories
- Some of the basic activities that are followed during software development by using the XP model are given below:

❖ **Coding:**

❖ **Testing:**

❖ **Listening**

❖ **Designing**

❖ **Feedback**

❖ **Simplicity**

❖ **Pair programming**

❖ **Continuous integration**

❖ **Refactoring**

❖ **Collective code ownership**

❖ **Planning game**

Advantages of Extreme Programming (XP)



- Slipped schedules
- Misunderstanding the business and/or domain
- Canceled projects
- Staff turnover
- Costs incurred in changes
- Business changes
- Production and post-delivery defects

Characteristics of Scrum

- Self-organize team
- Product progress in a series of month-long “Sprint”
- Requirement are captured as a items in a list of “product backlog”
- No specific engineering practices prescribed
- One of agile process

Different Roles in Scrum

In software development, three roles are defined in the scrum framework:

- ❖ Scrum team
- ❖ Scrum Master
- ❖ Product owner

Scrum Process

SCRUM PROCESS

Iterative development through short, time-boxed cycles called sprints, with regular inspection and adaptation to achieve project goals.



How Scrum Works ?

Scrum is based on a short iterations called Sprints, and emphasizes **inspection and adaptation**, allowing teams to improve continuously the product and the development process:

- **Product Owner**
- **Product Backlog,**
- **plans items from the Product Backlog**
- **Daily Standup**
- **Sprint Review.**

Advantages of Scrum

- Project deliverables quickly and efficiently.
- Ensures effective use of time and money.
- Large projects are divided into easily manageable sprints.
- Team gets clear visibility through Scrum meeting.
- Adopts feedback from customers and stakeholders.
- Individual effort of each team member is visible.
- Fast-moving development projects.

Disadvantages of Scrum

- Leads to scope creep, due to the lack of a definite end-date
- Chances of project failure are high if individuals aren't very committed
- Adopting the Scrum framework in large teams is challenging
- Successful only with experienced team members
- Daily meetings sometimes frustrate
- Team member leaves in the middle of a project, it can have a huge negative impact on the project